

توثيق طبقة الخدمات وطبقة الذكاء الاصطناعي في المشروع

هذه الوثيقة مكتوبة كرسالة تقنية تشرح المشروع من زاوية عملية، وتركز بشكل خاص على مجلد `core/services` لأنه يمثل قلب المنطق التحليلي والذكاء الاصطناعي داخل النظام.

المشروع: `django-Patient_data_analysis_system`

تاريخ التوثيق: 23-04-2026

البيئة الحالية: Python 3.13.3, Django 5.2.9, SQLite, Django REST Framework

رسالة افتتاحية

هذا المشروع ليس مجرد CRUD لإدارة بيانات المرضى، بل هو منصة مراقبة وبائية مصغرة تجمع بين تخزين الزيارات الطبية، تتبع البيانات الجغرافية، تقييم التجمعات المرضية، إصدار التنبيهات، وتكوين بيانات جاهزة للتعليم الآلي. أهم ما يميزه أن الذكاء الاصطناعي فيه ليس طبقة واحدة فقط، بل طبقة هجينة **Hybrid AI Layer** تجمع بين قواعد خبرائية، تحليل زمني، تحليل مكاني، وتجربة تعلم خاضع للإشراف لتقدير مستوى الخطورة.

عملياً، مجلد `core/services` هو المكان الذي تتحول فيه البيانات الخام مثل `Visit` و `GeoData` إلى قرارات تشغيلية مثل: هل لدينا تفشٍ محتمل؟ هل يجب إنشاء تقرير؟ هل يجب إنشاء hotspot؟ هل يوجد شذوذ زمني؟ وما هو مستوى خطر الزيارة إذا أردنا استخدام نموذج تنبؤ؟

فهرس مختصر

1. نظرة عامة على المشروع
2. التقنيات المستخدمة
3. طبقة الذكاء الاصطناعي
4. شرح ملفات `core/services` ملقاً
5. الأوامر المرتبطة بطبقة التحليل
6. سيناريوهات التشغيل الفعلية
7. نقاط القوة والحدود الحالية

نظرة عامة على المشروع

المشروع مبني حول كيان الزيارة الطبية `Visit` ، والمرض `Disease` ، وسجل الموقع `GeoData` ، والتقارير التشغيلية `Report` ، والعنقود الجغرافي `GeoCluster` . من هذه الكيانات يتم توليد رؤية تحليلية تساعد في اكتشاف الأنماط الوبائية محليًا.

الفكرة التشغيلية

- كل زيارة يمكن أن تكون لها بيانات جغرافية متعددة.
- المرض يمتلك خصائص شدة وخطورة وملف سياسة `policy_profile` .
- طبقة الخدمات تحلل الزيارة في سياق زمني ومكاني.
- ثم تنتج تنبيهًا، تقريرًا، hotspot، أو ملف تدريب لنموذج المخاطر.

الطبقات الأساسية

- `models.py` : تعريف البيانات والكيانات الأساسية.
- `serializers.py` : التحقق وتحويل البيانات بين JSON والنماذج.
- `views.py` : تعريف الـ API endpoints.
- `/services` : منطق الأعمال والتحليل والذكاء الاصطناعي.
- `signals.py` : تشغيل المراقبة تلقائيًا بعد حفظ البيانات الجغرافية.

التقنيات المستخدمة

- **Python 3.13.3**: لغة التنفيذ الأساسية.
- **Django 5.2.9**: إطار الويب وإدارة النماذج والـ ORM والإدارة.
- **Django REST Framework**: بناء واجهات الـ API.
- **django-filter**: دعم الفلاتر الزمنية والجغرافية على الزيارات والبيانات.
- **SimpleJWT**: توثيق يعتمد على JWT للوصول إلى الـ API.
- **SQLite**: قاعدة البيانات الحالية، مناسبة للتطوير والنمذجة السريعة.
- **JSON artifact**: تخزين نموذج المخاطر بصيغة JSON بدل مكتبات ML خارجية.
- **CSV export**: تجهيز بيانات AI وanalytics بصيغة قابلة لإعادة الاستخدام.

مهم جدًا: طبقة الذكاء الاصطناعي هنا لا تعتمد على TensorFlow أو PyTorch أو scikit-learn. هذه نقطة قوة وضعف في نفس الوقت: القوة أنها بسيطة وقابلة للفهم والتسليم الأكاديمي، والضعف أنها ليست منصة ML كبيرة أو مدربة بأسلوب بحثي متقدم.

طبقة الذكاء الاصطناعي في المشروع

طبقة الذكاء الاصطناعي هنا **هجين**، أي أنها لا تعتمد على خوارزمية واحدة فقط. بل تتكون من أربع عائلات منطقية:

- **ذكاء قائم على القواعد Rule-Based Intelligence:** ممثل أساسًا في `alert_policies.py` و `outbreak_engine.py`.
- **تحليل زمني Statistical Time Analysis:** ممثل في `trend.py` و `anomaly_detection.py`.
- **تحليل مكاني Spatial Intelligence:** ممثل في `spatial.py` و `cluster_service.py` و `dbscan_hotspots.py`.
- **تعلم خاضع للإشراف Lightweight Supervised Learning:** ممثل في `risk_prediction.py`.

لماذا هذه الطبقة مهمة؟

لأن النظام يحاول الإجابة عن أربعة أسئلة مختلفة:

1. هل الزيارة الحالية جزء من تفشي وبائي محلي؟
2. هل يوجد شذوذ زمني في عدد الحالات؟
3. هل يوجد hotspot جغرافي واضح؟
4. إذا جمعنا الخصائص كلها، فما فئة الخطر المتوقعة لهذه الزيارة؟

الخلاصة الذكية

المشروع أقرب إلى **نظام دعم قرار سريري-وبائي** منه إلى نموذج ذكاء اصطناعي أسود الصندوق. وهذا ممتاز إذا كان الهدف هو الشرح، الفهم، والقدرة على تبرير كل قرار اتخذته النظام.

شرح ملفات core/services ملفًا ملفًا

Re-export Layer

1. الملف `init__.py`

وظيفته: هذا الملف لا يحتوي خوارزمية بحد ذاته، لكنه يقوم بتصدير العقود الأساسية من ملف `contracts.py`.
الفائدة المعمارية: بدل أن تستورد بقية الطبقات الكائنات من `core.services.contracts` مباشرة، يمكنها الاستيراد من `core.services` وهذا يجعل الواجهة البرمجية أوضح وأنظف.
متى يظهر أثره؟ في `views.py` و `monitoring_service.py` عند استخدام `VisitOutbreakContext` أو `OutbreakAnalysis`.

Contracts

2. الملف `contracts.py`

الفكرة: هذا الملف يعرف اللغة الداخلية التي تتفاهم بها بقية ملفات الخدمات.

أهم الكيانات

- `VisitOutbreakContext` : يمثل الزيارة لحظة تحليلها، ويضم المرض، التاريخ، الإحداثيات، ونوع المنطقة.
- `NearbyCase` : يمثل حالة قريبة مكانيًا وزمنيًا من الزيارة الحالية.
- `TrendSnapshot` : يمثل مقارنة زمنية بين نافذة حالية ونافذة سابقة.
- `DiseaseAlertPolicy` : سياسة القرار الخاصة بكل مرض، وفيها أوزان وشروط التحليل.
- `OutbreakAnalysis` : الناتج النهائي لمحرك التفشي، ويشمل الدرجة، المستوى، الأسباب، وما إذا كان يجب إنشاء تقرير أو `cluster`.

لماذا هو مهم؟ لأنه يفصل بين تمثيل البيانات وبين طريقة جلبها أو تخزينها، وهذا يجعل الخدمات أقرب إلى منطق أعمال مستقل وليس كودًا متناثرًا داخل الـ `views`.

3. الملف `alert_policies.py`

هذا الملف هو قاعدة المعرفة الوبائية في النظام. هنا يتم تحديد كيفية التعامل مع كل مرض قبل تشغيل محرك القرار.

ماذا يحتوي؟

- خرائط alias بين الأكواد المختلفة مثل MEA و MEASLES و 1F03 .
- قوائم الأمراض عالية الأولوية والأمراض النادرة.
- قوالب سياسات جاهزة مثل high_priority و rare و cluster_sensitive .
- تخصيصات دقيقة لأمراض بعينها مثل cholera, polio, ebola, influenza.

كيف يعمل الكود؟

1. ينظف كود المرض ويحوّله لصيغة معيارية.
2. يحول أي alias إلى canonical code.
3. يطبّع infection_score إذا كانت القيم ليست أصلاً بين 0 و1.
4. يستنتج هل المرض نادر أو عالي الأولوية من خصائصه ومن policy_profile .
5. يبني سياسة أساسية DiseaseAlertPolicy انطلاقاً من شدة المرض وخطورته.
6. يطبق profile overrides ثم disease-specific overrides.

المغزى الذكي

هذه طبقة خبرائية. بدلاً من ترك كل الأمراض تتصرف بنفس الحساسية، يعطي النظام لكل مرض شخصية تشغيلية مختلفة. مثلاً المرض النادر قد يصبح critical من أول حالة، بينما مرض آخر يحتاج عددًا أكبر من الحالات أو نموًا زمنيًا واضحًا.

4. الملف `spatial.py`

وظيفته: تقديم المنطق الجغرافي الأساسي.

الدالة `distance_km`

تستخدم صيغة **Haversine** لحساب المسافة بين نقطتين على سطح الأرض بالكيلومترات. هذا مهم لأن الاعتماد على فرق درجات latitude/longitude مباشرة يعطي نتائج غير دقيقة.

```
distance = 2 * R * asin(sqrt(haversine_value))
```

الدالة `find_nearby_cases`

تقوم بالخطوات التالية:

1. تحدد نافذة زمنية تبدأ من `diagnosis_date - lookback_days`.
 2. تجلب كل `GeoData` لنفس المرض داخل هذه النافذة.
 3. تستثني الزيارة الحالية.
 4. تفلتر حسب `region_type` إذا طُلب ذلك.
 5. تحسب المسافة لكل مرشح.
 6. تحتفظ فقط بالحالات الواقعة داخل نصف القطر المحدد.
 7. تعيد قائمة مرتبة حسب المسافة والتاريخ.
- هذا الملف هو الأساس المكاني الذي تعتمد عليه لاحقًا ملفات التفشي و DBSCAN والعناقيد.

5. الملف trend.py

وظيفته: قياس الفرق بين فترة حالية وفترة سابقة لنفس المرض.

الدالة count_cases_for_window

- تعد الزيارات Visit لا سجلات GeoData .
- هذا يمنع over-counting إذا كانت الزيارة لها أكثر من سجل جغرافي.
- يمكن تقييد العد بحسب region_type .

الدالة build_trend_snapshot

تحسب:

- current_count : عدد الحالات في النافذة الحالية.
- previous_count : عدد الحالات في النافذة السابقة.
- growth_rate : نسبة التغير = $\text{previous} / (\text{current} - \text{previous})$.
- surge_ratio : كم مرة النافذة الحالية أكبر من السابقة = $\text{current} / \text{previous}$.

إذا لم توجد baseline سابقة وكان هناك نشاط حالي، فإن الملف يتعامل مع ذلك باعتباره نشاطاً جديداً ويعطي قيمًا موجبة بدل كسر الحساب بالقسمة على صفر.

6. الملف outbreak_engine.py

هذا هو أهم ملف ذكاء اصطناعي في المشروع. هو محرك اتخاذ القرار الخاص بالتفشي.

الفكرة العامة

يأخذ context و policy ، ثم يجمع بين ثلاثة أبعاد:

- شدة المرض وخصائصه.
- الكثافة المحلية للحالات المجاورة.
- النمو الزمني مقارنة بالنافذة السابقة.

الخطوات الداخلية

1. بناء نافذتين زمنيتين: previous و current.
2. استخدام find_nearby_cases لحساب الحالات القريبة.
3. استخدام build_trend_snapshot لحساب النمو والارتفاع.
4. تحويل النتائج إلى نقاط score عبر أوزان السياسة.
5. اختيار مستوى التنبيه النهائي.
6. بناء تفسير نصي reasons.
7. تحديد هل يجب إنشاء تقرير وهل يجب إنشاء cluster.

نظام النقاط

```
severity_points = 8 * severity_weight + 6 if high_priority
density_points = 20 * density_weight * density_factor
trend_points = 16 * trend_weight * trend_factor
rarity_points = 20 * rarity_weight if rare_disease
score = min(total, 100)
```

الدوال المساعدة المهمة

- density_factor_ : يحول عدد الحالات المحلية إلى عامل بين منخفض وعالي.
- trend_factor_ : يحول النمو الزمني إلى عامل scoring.
- determine_alert_level_ : يترجم score والسياق إلى low / medium / high / critical .
- build_reasons_ : يفسر القرار نصيًا.

القيمة الحقيقية لهذا الملف أنه لا يعطي نتيجة فقط، بل يعطي نتيجة **مفسّرة**. وهذا مهم جدًا في الأنظمة الصحية لأن التفسير أحيانًا أهم من الرقم نفسه.

Persistence

7. الملف `report_service.py`

وظيفته: تحويل ناتج التحليل إلى كيان `Report` داخل قاعدة البيانات.

الفلسفة التصميمية

هذا الملف لا يعيد تحليل الزيارة. التحليل تم بالفعل في `outbreak_engine.py`. هنا فقط يتم اتخاذ قرار الحفظ الذكي وعدم تكرار التقارير لنفس الحدث.

أهم ما يفعله

- يستخرج فترة التحليل من `analysis.metadata`.
- يبحث عن تقرير موجود لنفس الزيارة أو لنفس الحدث النشط زمنيًا.
- إذا لم يوجد، ينشئ تقريرًا جديدًا.
- إذا وجد، يحدّثه بدل مضاعفته.
- يضم الأسباب القديمة والجديدة عبر `merge_reasons_`.
- يرفع مستوى الخطورة إذا كانت الحالة الجديدة أخطر.

المغزى المعماري

هذا الملف يمنع تحول النظام إلى مولّد تقارير متكررة، ويحافظ على مفهوم أن عدة زيارات قد تنتمي إلى `outbreak` واحد مستمر، لا إلى 10 تقارير منفصلة بلا معنى.

Geo Persistence

8. الملف `cluster_service.py`

وظيفته: حفظ أو تحديث العنقود الجغرافي `GeoCluster` بناءً على نتيجة التحليل.

كيف يحسب cluster؟

1. يجلب إحدائيات الحالات المطابقة من `analysis.matched_case_ids`.
 2. يضيف إليها إحدائيات الحالة الحالية.
 3. يحسب المركز الجغرافي المتوسط `center_lat` و `center_long`.
 4. يحسب نصف القطر كأبعد مسافة عن هذا المركز.
 5. يحوّل `alert_level` إلى `risk_level`.
 6. إذا وجد cluster قريبًا خلال آخر 30 يومًا، يحدّثه بدل إنشاء واحد جديد.
- هذه خدمة تشغيلية مهمة لأنها تمثل "الطبقة المرئية" للخطر على الخريطة.

Orchestration

9. الملف `monitoring_service.py`

وظيفته: تشغيل كامل pipeline المراقبة على سجل `GeoData` واحد.

تسلسل العمل

1. بناء `VisitOutbreakContext` من الـ `GeoData`.
 2. جلب سياسة المرض عبر `get_policy_for_disease`.
 3. تشغيل `evaluate_visit_outbreak`.
 4. إنشاء تقرير إذا أمكن.
 5. إنشاء cluster إذا أمكن.
 6. إرجاع نتيجة منظمة تحتوي على `context` و `policy` و `analysis` و `persistence`.
- هذه الخدمة هي orchestrator حقيقي، ولذلك هي مرتبطة مباشرة بـ `signals.py` لتفعيل المراقبة تلقائيًا بعد حفظ بيانات الموقع.

10. الملف `anomaly_detection.py`

هذا الملف يكتشف الشذوذ الزمني في النشاط المرضي.

الفكرة الإحصائية

بدل تحليل زيارة واحدة، هذا الملف ينظر إلى السلسلة الزمنية اليومية لكل مرض ولكل نوع منطقة `region_type` ، ثم يسأل: هل عدد الحالات في يوم معين شاذ بشكل واضح مقارنة بال `baseline` القريب منه؟

كيف يبني السلسلة؟

- يجمع عدد الحالات يوميًا من `GeoData` مع عد `distinct` للزيارات.
- يفصل السلاسل حسب الزوج `(disease_id, region_type)` .
- يملأ الأيام المفقودة بصفر حتى لا تنكسر السلسلة الزمنية.

المؤشرات المستخدمة

- `baseline_mean` : متوسط القيم السابقة.
- `baseline_median` : الوسيط.
- `baseline_max` : أعلى قيمة تاريخية في النافذة.
- `surge_ratio` : نسبة الارتفاع مقارنة بالمتوسط.
- `deviation_score` : درجة الانحراف باستخدام **MAD**.

معادلة الانحراف

$$\text{deviation_score} = 0.6745 * (\text{observed} - \text{baseline_median}) / \text{MAD}$$

استخدام **Median Absolute Deviation** مهم لأنه أكثر مقاومة للقيم الشاذة من الاعتماد على `mean/std` فقط.

حساب `anomaly_score`

- إذا كان `baseline` صفرًا والنشاط الجديد تجاوز العتبة الدنيا، يحصل على نقاط بداية.
- تضاف نقاط من الفرق عن المتوسط.
- تضاف نقاط من `surge_ratio` .
- تضاف نقاط من `deviation_score` .
- تضاف نقاط إضافية إذا كان المرض عالي الأولوية أو نادرًا.

ثم يتم تحويل `anomaly_score` إلى `low / medium / high / critical` ، وتصفية النتائج الضعيفة غير المهمة.

هذا الملف من أكثر الملفات "ذكاء" في المشروع لأنه لا يكتفي بعدد الحالات، بل يقارنها بسلوك زمني تاريخي ويستخرج شذوذاً meaningful.

AI Layer

Spatial Clustering

11. الملف dbscan_hotspots.py

هذا الملف يطبق خوارزمية DBSCAN-like clustering بطريقة يدوية.

لماذا DBSCAN؟

لأنها خوارزمية مناسبة لاكتشاف التجمعات المكانية من نقاط غير معروفة العدد مسبقاً، وتستطيع فصل الضوضاء noise عن clusters الفعلية.

كيف نُغذت هنا؟

1. جمع النقاط من GeoData داخل نافذة زمنية أو نطاق تاريخ.
2. تقسيم النقاط حسب المرض.
3. لكل نقطة، تنفيذ region_query للبحث عن الجيران داخل eps_km .
4. إذا كان عدد الجيران أقل من min_samples تصنف كنقطة ضوضاء.
5. إذا استوفت الشرط، يبدأ توسيع cluster عبر seeds مثل DBSCAN الكلاسيكية.
6. بعد اكتمال الأعضاء، يتم حساب مركز cluster ونصف قطره وتاريخ بدايته ونهايته.

ملاحظات مهمة عن التطبيق

- التطبيق هنا يدوي وليس من مكتبة خارجية.
- التعقيد الحسابي أقرب إلى $O(n^2)$ لأن البحث عن الجيران يمر على كل النقاط.
- هذا مقبول لبيانات صغيرة أو متوسطة، لكنه يحتاج spatial index لو كبر الحجم.

الحفظ

بعد الاكتشاف، يمكن تمرير النتائج إلى persist_dbscan_clusters الذي يدمجها مع clusters قريبة موجودة بدل إنشاء duplicates كثيرة.

12. الملف dataset_export.py

وظيفته: استخراج بيانات جاهزة للتحليل أو التدريب بصيغتين رئيسيتين.

الملف الأول: visit_level_dataset.csv

كل صف يمثل زيارة واحدة ويحتوي features مثل:

- بيانات المريض: الجنس، العمر.
- بيانات الزيارة: التاريخ، الحالة الاجتماعية، الوزن، الطول، BMI.
- بيانات المرض: نوعه، policy_profile ، infection_score ، risk_level .
- بيانات جغرافية: primary region, geodata count, home/work flags.
- بيانات المختبر: عدد الفحوص وعدد النتائج غير الطبيعية.
- بيانات التقارير: آخر alert level وآخر risk score وعدد التنبيهات النشطة.

الملف الثاني: disease_region_day_dataset.csv

كل صف يمثل يومًا + مرضًا + نوع منطقة، ويجمع metrics مثل:

- عدد الزيارات والمرضى والأطباء والمشافي.
- المتوسطات الجغرافية للإحداثيات.
- عدد الفحوص المختبرية غير الطبيعية.
- عدد التنبيهات النشطة عالية ومتوسطة ودرجة.
- عدد الـ hotspots النشطة.

هذا الملف مهم جدًا لأنه يقوم بعملية **feature engineering** فعلية، أي تحويل البيانات التشغيلية إلى تمثيل يصلح للتعلم الآلي والـ dashboards والبحوث.

13. الملف `disease_reference.py`

وظيفته: تصنيف الأمراض المرجعية المستوردة وإعطاؤها profile تشغيليًا.

لماذا نحتاجه؟

لأن استيراد الأمراض من CSV وحده لا يكفي. يجب أن تتحول هذه الأمراض إلى معلومات تشغيلية تساعد طبقة الذكاء الاصطناعي لاحقًا.

كيف يصنف المرض؟

- إذا كان الكود ضمن الأمراض النادرة، يصبح `policy_profile = rare`.
- إذا كان ضمن الأمراض البيئية التنفسية، يصبح `environmental_signal`.
- إذا كان مرضًا معويًا أو food/waterborne، يصبح `cluster_sensitive`.
- إذا كان تنفسيًا أو respiratory-driven، يصبح `surge_sensitive`.
- إذا كانت الخطورة مرتفعة جدًا، قد يصبح `high_priority`.

هذا الملف يربط بين taxonomy الطبية وبين سلوك المحرك الذكي. بمعنى آخر: هو layer ترجمة من "بيانات مرجعية" إلى "سياسة تشغيل".

14. الملف `risk_prediction.py`

هذا الملف يمثل تجربة التعلم الخاضع للإشراف داخل المشروع.

الفكرة العامة

بدل استخدام مكتبة ML خارجية، تم بناء نموذج baseline بسيط وقابل للشرح يعتمد على **Nearest Centroid Classification**.

كيف تُبنى الـ features؟

الدالة `build_visit_feature_map` تبني vector للزيارة يضم:

- شدة المرض وخطورة العدوى.
- أعلام `high_priority` و `rare`.
- العمر و BMI.
- عدد سجلات GeoData ونوع المنطقة الأساسية.
- عدد الاختبارات غير الطبيعية.
- آخر `risk score` وعدد التنبيهات النشطة.
- المسافة لأقرب hotspot وهل الزيارة قريبة من hotspot.
- تمثيل one-hot لـ `policy_profile`.

من أين تأتي الـ labels؟

1. أولاً من التقارير الموجودة إذا كانت هناك تقارير مرتبطة بالزيارة.
 2. إن لم توجد تقارير، يتم توليد label من `outbreak_engine` نفسه.
- هذا يعني أن النموذج يتعلم جزئياً من الخبرة التشغيلية المخزنة، وجزئياً من المحرك القاعدي الحالي.

كيف يتم التدريب؟

1. تجميع كل الزيارات وتحويلها إلى `samples`.
2. تطبيع كل feature بطريقة `min-max normalization`.
3. حساب centroid لكل label من labels الأربع: `low, medium, high, critical`.
4. حفظ النتائج في ملف JSON يحتوي `feature bounds` و `class centroids`.

كيف يتم التنبؤ؟

1. بناء feature vector جديد للزيارة.
2. تطبيعه بنفس حدود `training`.

3. حساب Euclidean distance بينه وبين centroid كل فئة.

4. اختيار الفئة ذات أصغر مسافة.

5. حساب confidence تقريبية عبر inverse distance weighting.

```
predicted_label = argmin(distance_to_centroid)
confidence = inverse_distance(predicted) / sum(all_inverse_distances)
```

لماذا هذا النموذج مهم؟

لأنه بسيط جدًا، مفهوم جدًا، ومناسب لمشروع أكاديمي أو نسخة أولية من النظام. كل prediction هنا قابل للتفسير، ويمكن عرض المسافات لكل class بدل إعطاء label بلا تفسير.

لكن ما حدوده؟

- ليس فيه train/test split داخل الخدمة نفسها.
- ليس فيه tuning أو cross-validation.
- تعقيده منخفض جدًا مقارنة بنماذج ML الاحترافية.
- يعتمد على جودة labels المولدة من التقارير أو من المحرك القاعدي.

الأوامر الإدارية المرتبطة بطبقة التحليل

إلى جانب ملفات الخدمات، يوجد عدد من أوامر `management commands` التي تشغل الطبقة الذكية عملياً:

- `python manage.py import_diseases --csv-path ...` : استيراد الأمراض المرجعية وتصنيفها.
- `python manage.py export_ai_dataset --output-dir ...` : تصدير datasets جاهزة للتدريب أو البحث.
- `python manage.py train_risk_model --output-path ...` : تدريب نموذج المخاطر baseline.
- `python manage.py predict_risk --visit-id ... --model-path ...` : التنبؤ بمستوى الخطر لزيارة معينة.
- `python manage.py run_anomaly_detection ...` : استخراج الشذوذات الزمنية.
- `python manage.py run_dbscan_hotspots ...` : استخراج hotspots مكانية بأسلوب DBSCAN.

وجود هذه الأوامر يعني أن طبقة الذكاء الاصطناعي ليست محصورة فقط داخل API، بل يمكن تشغيلها batch أيضاً من سطر الأوامر.

سيناريوهات التشغيل الفعلية

1. سيناريو المراقبة التلقائية بعد حفظ GeoData

1. يحفظ النظام سجل `GeoData`.
2. يشغل `signals.py` خدمة `run_monitoring_for_geodata` بعد `transaction.on_commit`.
3. تُبنى السياسة والسياق.
4. يعمل محرك التفشي.
5. قد يُنشأ تقرير و/أو cluster تلقائياً.

2. سيناريو تقييم outbreak يدوياً من الـ API

1. يطلب العميل `/api/visits/<id>/evaluate_outbreak/`.
2. تختار الواجهة GeoData المناسب.
3. تجلب السياسة المناسبة للمرض.
4. يعاد للمستخدم `policy + analysis + persistence status`.

3. سيناريو التنبؤ بالمخاطر

1. تصدير أو تجهيز البيانات الحالية.
2. تدريب نموذج baseline.
3. طلب `/api/visits/<id>/predict_risk/` أو أمر `predict_risk`.

4. إرجاع predicted label + confidence + feature values + class distances.

نقاط القوة والحدود الحالية

نقاط القوة

- فصل ممتاز بين الواجهة والخدمات.
- طبقة AI قابلة للفهم والتفسير.
- وجود تحليل مكاني وزمني وتعلم خاضع للإشراف في مشروع واحد.
- إمكانية التشغيل API + commands + signals.
- وجود datasets جاهزة للتصدير والبحث.

الحدود الحالية

- SQLite جيد للبروتوتايب لكنه ليس الأفضل للأحمال الكبيرة.
- حسابات DBSCAN والجوار تعتمد على loops وقد تصبح أبطأ مع البيانات الكبيرة.
- نموذج المخاطر baseline وليس نموذجًا بحثيًا متقدمًا.
- الذكاء القاعدي يعتمد بقوة على جودة disease metadata.
- لا توجد حاليًا spatial indexes أو PostgreSQL/PostGIS.

الخاتمة

لو أردت وصف المشروع بجملة واحدة، فالوصف الأقرب هو: **نظام مراقبة وبائية مبني على Django، يستخدم طبقة خدمات منظمة لتطبيق ذكاء هجين يجمع بين القواعد الخبرائية، التحليل الزمني، التحليل المكاني، والتنبؤ المبسط بالمخاطر.**

وأهم ملفين لشرح "الذكاء" فيه هما `alert_policies.py` لأنه يمثل المعرفة الطبية/التشغيلية، و `outbreak_engine.py` لأنه يحول هذه المعرفة إلى قرار. ثم تأتي فوقهما طبقات مكملية: `anomaly_detection.py` للشذوذ الزمني، `dbscan_hotspots.py` للتجمعات المكانية، و `risk_prediction.py` للتنبؤ الإشرافي.

إذا احتجت لاحقًا، يمكن بسهولة تحويل هذه الوثيقة إلى نسخة ثانية أكثر أكاديمية، أو نسخة موجهة للعرض الشفهي، أو نسخة مختصرة لعرض التخرج.

تم إعداد هذه الوثيقة بناءً على الحالة الحالية للملفات الموجودة في `src/core/services` وملفات الـ API والإعدادات المرتبطة بها داخل المشروع.